

Lazy Evaluation

CS 350

Dr. Joseph Eremondi

March 16, 2025

Lazy Evaluation

Lazy Evaluation

- Objectives

- Objectives
 - To understand the design space of evaluation order

- Objectives
 - To understand the design space of evaluation order
- Key Concepts

- Objectives
 - To understand the design space of evaluation order
- Key Concepts
 - Lazy evaluation

- Objectives
 - To understand the design space of evaluation order
- Key Concepts
 - Lazy evaluation
 - Strict evaluation

- Objectives
 - To understand the design space of evaluation order
- Key Concepts
 - Lazy evaluation
 - Strict evaluation
 - Strictness points

- Objectives
 - To understand the design space of evaluation order
- Key Concepts
 - Lazy evaluation
 - Strict evaluation
 - Strictness points
 - Thunks

The Type of Substitution

- We could replace a variable with *any expression* in substitution

The Type of Substitution

- We could replace a variable with *any expression* in substitution
 - Doesn't have to be a value

The Type of Substitution

- We could replace a variable with *any expression* in substitution
 - Doesn't have to be a value

The Type of Substitution

- We could replace a variable with *any expression* in substitution
 - Doesn't have to be a value

```
(define (subst [toReplace : Symbol]  
              [replacedBy : Exp]  
              [replaceIn : Exp]) : Exp ....)
```

Substituting Expressions vs. Values

Substituting Expressions vs. Values

```
(define (interp expr)
  (type-case Exp interp
    ....
    [(appE fun arg)
     (type-case Value (interp fun)
       [(lamV var body)
        (let* ([argExp (Value->Exp (interp arg))]
               [subbedBody (subst var argExp body)])
          (interp subbedBody))]
       [else
        (error 'interp "Tried to call non-function")]]])])
```

- We evaluate the argument before substituting it into the function body

Substituting Expressions vs. Values

```
(define (interp expr)
  (type-case Exp interp
    ....
    [(appE fun arg)
     (type-case Value (interp fun)
       [(lamV var body)
        (let* ([argExp (Value->Exp (interp arg))]
               [subbedBody (subst var argExp body)])
          (interp subbedBody))]
       [else
        (error 'interp "Tried to call non-function")]]])
```

- We evaluate the argument before substituting it into the function body
 - Why do we have to do this?

Substituting Expressions vs. Values

```
(define (interp expr)
  (type-case Exp interp
    ....
    [(appE fun arg)
     (type-case Value (interp fun)
       [(lamV var body)
        (let* ([argExp (Value->Exp (interp arg))]
               [subbedBody (subst var argExp body)])
          (interp subbedBody))]
       [else
        (error 'interp "Tried to call non-function")]]])])
```

- We evaluate the argument before substituting it into the function body
 - Why do we have to do this?
 - We don't

Substitution-Based Lazy Evaluation

Substitution-Based Lazy Evaluation

```
(define (interp expr)
  (type-case Exp interp
    ....
    [(appE fun arg)
     (type-case Value (interp fun)
       [(lamV var body)
        (let* ([subbedBody (subst var arg body)])
          (interp subbedBody))]
       [else
        (error 'interp "Tried to call non-function")]]])])
```

- Don't interpret argument to a value before substituting it

Substitution-Based Lazy Evaluation

```
(define (interp expr)
  (type-case Exp interp
    ....
    [(appE fun arg)
     (type-case Value (interp fun)
       [(lamV var body)
        (let* ([subbedBody (subst var arg body)])
          (interp subbedBody))]
       [else
        (error 'interp "Tried to call non-function")]]])])
```

- Don't interpret argument to a value before substituting it
- Works just as well, *but we get different behaviour*

Classifying Languages

- Many ways to classify programming languages

Classifying Languages

- Many ways to classify programming languages
- Sometimes presence/absence of features

Classifying Languages

- Many ways to classify programming languages
- Sometimes presence/absence of features
 - e.g. has lambda, mutable variables, objects, etc.

Classifying Languages

- Many ways to classify programming languages
- Sometimes presence/absence of features
 - e.g. has lambda, mutable variables, objects, etc.
- Others, there are *design choices* which every language must make

Classifying Languages

- Many ways to classify programming languages
- Sometimes presence/absence of features
 - e.g. has lambda, mutable variables, objects, etc.
- Others, there are *design choices* which every language must make
 - Lazy vs. Strict is one

Classifying Languages

- Many ways to classify programming languages
- Sometimes presence/absence of features
 - e.g. has lambda, mutable variables, objects, etc.
- Others, there are *design choices* which every language must make
 - Lazy vs. Strict is one
 - We'll see more

Strict vs. Lazy Semantics

- **Strict evaluation order** is when we evaluate function arguments to a value before substituting them into the function body

Strict vs. Lazy Semantics

- **Strict evaluation order** is when we evaluate function arguments to a value before substituting them into the function body
 - Also called **eager** semantics

Strict vs. Lazy Semantics

- **Strict evaluation order** is when we evaluate function arguments to a value before substituting them into the function body
 - Also called **eager** semantics
- **Lazy evaluation order** is when we don't evaluate function parameters before substituting them into function bodies

Differences between Lazy and Strict

- In a language with no side effects:

Differences between Lazy and Strict

- In a language with no side effects:
 - Lazy and strict produce *the exact same results* with strict and lazy semantics

Differences between Lazy and Strict

- In a language with no side effects:
 - Lazy and strict produce *the exact same results* with strict and lazy semantics
- Difference is with side-effects

Differences between Lazy and Strict

- In a language with no side effects:
 - Lazy and strict produce *the exact same results* with strict and lazy semantics
- Difference is with side-effects
 - Lazy: some code might run later

Differences between Lazy and Strict

- In a language with no side effects:
 - Lazy and strict produce *the exact same results* with strict and lazy semantics
- Difference is with side-effects
 - Lazy: some code might run later
 - Or not run at all

Differences between Lazy and Strict

- In a language with no side effects:
 - Lazy and strict produce *the exact same results* with strict and lazy semantics
- Difference is with side-effects
 - Lazy: some code might run later
 - Or not run at all
- If strict succeeds, lazy succeeds

Differences between Lazy and Strict

- In a language with no side effects:
 - Lazy and strict produce *the exact same results* with strict and lazy semantics
- Difference is with side-effects
 - Lazy: some code might run later
 - Or not run at all
- If strict succeeds, lazy succeeds
- Sometimes, strict fails but lazy succeeds

Example: Error

Example: Error

```
{let1 f {lam x {lam y {if {zero? x} y x}}}  
      {{f #t} {+ 3 #t}}}
```

- Strict evaluation:

Example: Error

```
{let1 f {lam x {lam y {if {zero? x} y x}}}  
      {{f #t} {+ 3 #t}}}}
```

- Strict evaluation:
 - Evaluates `{+ 3 #t}` before substituting it in the function body

Example: Error

```
{let1 f {lam x {lam y {if {zero? x} y x}}}  
      {{f #t} {+ 3 #t}}}
```

- Strict evaluation:
 - Evaluates `{+ 3 #t}` before substituting it in the function body
 - Raises an error because it's undefined

Example: Error

```
{let1 f {lam x {lam y {if {zero? x} y x}}}  
      {{f #t} {+ 3 #t}}}
```

- Strict evaluation:
 - Evaluates `{+ 3 #t}` before substituting it in the function body
 - Raises an error because it's undefined
- Lazy: no error

Example: Error

```
{let1 f {lam x {lam y {if {zero? x} y x}}}  
      {{f #t} {+ 3 #t}}}
```

- Strict evaluation:
 - Evaluates `{+ 3 #t}` before substituting it in the function body
 - Raises an error because it's undefined
- Lazy: no error
 - Replaces `y` with `{+ 3 #t}` *without evaluating*

Example: Error

```
{let1 f {lam x {lam y {if {zero? x} y x}}}  
      {{f #t} {+ 3 #t}}}
```

- Strict evaluation:
 - Evaluates `{+ 3 #t}` before substituting it in the function body
 - Raises an error because it's undefined
- Lazy: no error
 - Replaces `y` with `{+ 3 #t}` *without evaluating*
 - When we substitute, it only ends up in the branch of the `if` that we don't take

Example: Loop

Example: Loop

```
{let1 f {lam x {lam y {if {zero? x} y {* x 2}}}}
  {{f 3}
   {{lam x {x x}} {lam x {x x}}}
  }}
```

- `{{lam x {x x}} {lam x {x x}}}` runs forever

Example: Loop

```
{let1 f {lam x {lam y {if {zero? x} y {* x 2}}}}
  {{f 3}
   {{lam x {x x}} {lam x {x x}}}
  }}
```

- `{{lam x {x x}} {lam x {x x}}}` runs forever
- Strict evaluation:

Example: Loop

```
{let1 f {lam x {lam y {if {zero? x} y {* x 2}}}}
  {{f 3}
   {{lam x {x x}} {lam x {x x}}}
  }}
```

- `{{lam x {x x}} {lam x {x x}}}` runs forever
- Strict evaluation:
 - Evaluates the argument that runs forever

Example: Loop

```
{let1 f {lam x {lam y {if {zero? x} y {* x 2}}}}
  {{f 3}
   {{lam x {x x}} {lam x {x x}}}
  }}
```

- `{{lam x {x x}} {lam x {x x}}}` runs forever
- Strict evaluation:
 - Evaluates the argument that runs forever
 - never gets to substituting

Example: Loop

```
{let1 f {lam x {lam y {if {zero? x} y {* x 2}}}}
  {{f 3}
   {{lam x {x x}} {lam x {x x}}}
  }}
```

- `{{lam x {x x}} {lam x {x x}}}` runs forever
- Strict evaluation:
 - Evaluates the argument that runs forever
 - never gets to substituting
- Terminates with lazy evaluation

Example: Loop

```
{let1 f {lam x {lam y {if {zero? x} y {* x 2}}}}
  {{f 3}
   {{lam x {x x}} {lam x {x x}}}
  }}
```

- `{{lam x {x x}} {lam x {x x}}}` runs forever
- Strict evaluation:
 - Evaluates the argument that runs forever
 - never gets to substituting
- Terminates with lazy evaluation
 - Never evaluate the function calling itself

Example: Loop

```
{let1 f {lam x {lam y {if {zero? x} y {* x 2}}}}
  {{f 3}
   {{lam x {x x}} {lam x {x x}}}
  }}
```

- `{{lam x {x x}} {lam x {x x}}}` runs forever
- Strict evaluation:
 - Evaluates the argument that runs forever
 - never gets to substituting
- Terminates with lazy evaluation
 - Never evaluate the function calling itself
 - When substitute, it only ends up in the branch of the `if` that we don't take

The General Rule

- For function calls

The General Rule

- For function calls
 - Lazy: substitute the argument *expression* into the body, then evaluate the body

The General Rule

- For function calls
 - Lazy: substitute the argument *expression* into the body, then evaluate the body
 - Strict: evaluate the argument to a value, then substitute into the body, then evaluate the body

Non-Function Recursion

- Recall: with recursion, we needed to have all recursive references in the body of a lambda

Non-Function Recursion

- Recall: with recursion, we needed to have all recursive references in the body of a lambda
- With lazy evaluation, we don't need this, because we won't try to evaluate the variable

Non-Function Recursion

- Recall: with recursion, we needed to have all recursive references in the body of a lambda
- With lazy evaluation, we don't need this, because we won't try to evaluate the variable
- Lets us define **infinite data structures** using recursion

Non-Function Recursion

- Recall: with recursion, we needed to have all recursive references in the body of a lambda
- With lazy evaluation, we don't need this, because we won't try to evaluate the variable
- Lets us define **infinite data structures** using recursion
 - As long as we only ever access a finite part of the structure, program will terminate

Non-Function Recursion

- Recall: with recursion, we needed to have all recursive references in the body of a lambda
- With lazy evaluation, we don't need this, because we won't try to evaluate the variable
- Lets us define **infinite data structures** using recursion
 - As long as we only ever access a finite part of the structure, program will terminate
 - e.g. "the list of all natural numbers"

Non-Function Recursion

- Recall: with recursion, we needed to have all recursive references in the body of a lambda
- With lazy evaluation, we don't need this, because we won't try to evaluate the variable
- Lets us define **infinite data structures** using recursion
 - As long as we only ever access a finite part of the structure, program will terminate
 - e.g. "the list of all natural numbers"
- See Racket examples in `lazy.rkt`

Strictness Points

- Some features *had* to evaluate their arguments before using them

Strictness Points

- Some features *had* to evaluate their arguments before using them
 - e.g. the number we're testing in `if`

Strictness Points

- Some features *had* to evaluate their arguments before using them
 - e.g. the number we're testing in `if`
 - Can't progress until it's a value

Strictness Points

- Some features *had* to evaluate their arguments before using them
 - e.g. the number we're testing in `if`
 - Can't progress until it's a value
 - These are *strictness points*

Strictness Points

- Some features *had* to evaluate their arguments before using them
 - e.g. the number we're testing in `if`
 - Can't progress until it's a value
 - These are *strictness points*
 - Places an un-evaluated argument will be evaluated

Strictness Points

- Some features *had* to evaluate their arguments before using them
 - e.g. the number we're testing in `if`
 - Can't progress until it's a value
 - These are *strictness points*
 - Places an un-evaluated argument will be evaluated
- Include

Strictness Points

- Some features *had* to evaluate their arguments before using them
 - e.g. the number we're testing in `if`
 - Can't progress until it's a value
 - These are *strictness points*
 - Places an un-evaluated argument will be evaluated
- Include
 - Boolean tested in `if`

Strictness Points

- Some features *had* to evaluate their arguments before using them
 - e.g. the number we're testing in `if`
 - Can't progress until it's a value
 - These are *strictness points*
 - Places an un-evaluated argument will be evaluated
- Include
 - Boolean tested in `if`
 - Arguments to plus or times

Strictness Points

- Some features *had* to evaluate their arguments before using them
 - e.g. the number we're testing in `if`
 - Can't progress until it's a value
 - These are *strictness points*
 - Places an un-evaluated argument will be evaluated
- Include
 - Boolean tested in `if`
 - Arguments to `plus` or `times`
 - The *function* in a function call

Strictness Points

- Some features *had* to evaluate their arguments before using them
 - e.g. the number we're testing in `if`
 - Can't progress until it's a value
 - These are *strictness points*
 - Places an un-evaluated argument will be evaluated
- Include
 - Boolean tested in `if`
 - Arguments to plus or times
 - The *function* in a function call
 - not the argument

Laziness in real languages

- Python and JavaScript have *generators*

Laziness in real languages

- Python and JavaScript have *generators*
 - A structure we can iterate from, that doesn't generate the entire list we're going through

Laziness in real languages

- Python and JavaScript have *generators*
 - A structure we can iterate from, that doesn't generate the entire list we're going through
 - This is basically just a lazy list, implemented similar way

Laziness in real languages

- Python and JavaScript have *generators*
 - A structure we can iterate from, that doesn't generate the entire list we're going through
 - This is basically just a lazy list, implemented similar way
- Can probably do something similar with iterators in C++

- We can simulate laziness in an eager language using just functions

Laziness via lambda

- We can simulate laziness in an eager language using just functions
- A function that ignores its argument acts like a lazy value

Laziness via lambda

- We can simulate laziness in an eager language using just functions
- A function that ignores its argument acts like a lazy value
 - To force it back to a value, you call it with any value

Laziness via lambda example

Laziness via lambda example

```
{let1 f {lam x {lam y {if {zero? z} {y 0} {* x 2}}}}  
      {{f 3} {lam {x} {+ x #t}}}}
```

- Plus of boolean behind a lambda, so no error

Laziness via lambda example

```
{let1 f {lam x {lam y {if {zero? z} {y 0} {* x 2}}}}  
      {{f 3} {lam {x} {+ x #t}}}}
```

- Plus of boolean behind a lambda, so no error
 - Function body stored as *expression*

Laziness via lambda example

```
{let1 f {lam x {lam y {if {zero? z} {y 0} {* x 2}}}}  
      {{f 3} {lam {x} {+ x #t}}}}
```

- Plus of boolean behind a lambda, so no error
 - Function body stored as *expression*
 - Never run unless function called

Laziness via lambda example

```
{let1 f {lam x {lam y {if {zero? z} {y 0} {* x 2}}}}  
      {{f 3} {lam {x} {+ x #t}}}}
```

- Plus of boolean behind a lambda, so no error
 - Function body stored as *expression*
 - Never run unless function called
 - Only called in un-taken if branch

Laziness via lambda example

```
{let1 f {lam x {lam y {if {zero? z} {y 0} {* x 2}}}}  
      {{f 3} {lam {x} {+ x #t}}}}
```

- Plus of boolean behind a lambda, so no error
 - Function body stored as *expression*
 - Never run unless function called
 - Only called in un-taken if branch
- In strict language, has the same behaviour as the lazy version